

Reviewer Pack — Minimal Rank of Quotient Representations vs Boolean Function...

Entry b9210c441b90 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 10:34:51 UTC

Minimal Rank of Quotient Representations vs Boolean Function Entropic Complexity

Entry ID: b9210c441b90 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 10:34:51 UTC

1. Conjecture

Field A (mathematical branch): Module Theory (Quotient Representations) **Field B** (complexity object): Complexity Theory: Entropy-based Communication Complexity

Statement:

[‘For any n -ary boolean function f , the minimal rank of its quotient representation over $GF(2)$ is upper bounded by a function of the entropic complexity of f .’, ‘Equivalently, there exists a constant c such that for all boolean functions f , $\text{Rank}_{\text{quot}}(f) \leq c * \text{Entropy_complexity}(f)$.’, ‘For all instances with property P , property Q holds.’]

Rationale (proposer’s reasoning):

[‘Quotient representations provide a way to study the algebraic structure of boolean functions. Entropic complexity measures the information content of a function, which is relevant to communication complexity.’, ‘This conjecture suggests that these two concepts are tightly connected, potentially revealing a deep connection between algebra and information theory.’, ‘If true, it would offer a new perspective on understanding the limits of communication complexity.’]

Taxonomy category: module_theory_to_entropy_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2cfaf7e61084c132

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a boolean function f , the quotient representation rank and entropic complexity correlation is within bounds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal rank" [quotient representations] AND [boolean function] entropic complexity - "rank_quot(f)" <= c * "Entropy_complexity(f)" - property P -> property Q

Top relevant hits considered: - [http://arxiv.org/abs/0911.3482v5] Complexity of Networks (reprise) - [http://arxiv.org/abs/1810.08668v2] Parity Decision Tree Complexity is Greater Than Granularity - [http://arxiv.org/abs/1501.01331v1] DNF complexity of complete boolean functions - [http://arxiv.org/abs/2604.05712v1] Precise measurement of the CKM angle γ with a novel approach - [http://arxiv.org/abs/2604.05701v1] Measurement of the CKM angle γ in $B^\pm \rightarrow D(\rightarrow K_S^0 h'^+ h'^-) h^\pm$ dec - [http://arxiv.org/abs/2407.14261v2] Study of charmonium production via the decay to $p\bar{p}$ at $\sqrt{s} = 13TeV$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_quotient_representation(f):
        n = int(math.log2(len(f)))
        A = [[f[i * (1 << (n - j)) + k] for k in range(1 << j)] for j in range(n)]
        rank = 0
        for row in A:
            if any(row):
                rank += 1
                for i in range(len(A)):
                    if A[i][0]:
                        for j in range(len(row)):
                            A[i][j] ^= row[j]
        return rank

    def compute_entropy_complexity(f):
        n = int(math.log2(len(f)))
        counts = [f.count(i) for i in range(2)]
        entropy = 0
```

```

    for count in counts:
        if count > 0:
            p = count / len(f)
            entropy -= p * math.log2(p)
    return entropy

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    f = generate_boolean_function(n)
    rank_quot = compute_quotient_representation(f)
    entropy_complexity = compute_entropy_complexity(f)

    results.append({
        "n": n,
        "rank_quot": rank_quot,
        "entropy_complexity": entropy_complexity
    })

mean_rank_quot = sum(result["rank_quot"] for result in results) / len(results)
mean_entropy_complexity = sum(result["entropy_complexity"] for result in results) / len(results)

if all(result["rank_quot"] <= 2 * result["entropy_complexity"] for result in results):
    conjecture_holds = True
    counterexample = ""
else:
    conjecture_holds = False
    counterexample = "Rank_quot exceeds 2 * Entropy_complexity"

return {
    "metric_name": "rank_quot vs entropy_complexity",
    "metric_value": mean_rank_quot,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank_quot = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank_quot} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank_quot} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)

```

```
print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\") first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9cbe65da.py", line 84, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9cbe65da.py", line 52, in run_trial
    rank_quot = compute_quotient_representation(f)
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9cbe65da.py", line 26, in compute_quoti
    A = [[f[i * (1 << (n - j)) + k] for k in range(1 << j)] for j in range(n)]
          ^
```

UnboundLocalError: cannot access local variable 'i' where it is not associated with a value

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents a definitive assessment of the conjecture. | next: Investigate and fix the crash in the test code to allow for a proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10961
2	propose	ollama_remote	glm4:latest	0	0	9796
3	preregistration	ollama_remote	glm4:latest	0	0	5519
4	novelty	ollama_remote	glm4:latest	0	0	4682
5	novelty	ollama_remote	glm4:latest	0	0	7348
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14320
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13073
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7372
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9803
10	judge	ollama_remote	glm4:latest	0	0	23638

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 106513 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/b9210c441b90.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/b9210c441b90.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/b9210c441b90.tar.gz (if generated)